

Storage-Triggered Leader Step-Down in a Raft-Based Key-Value Store

Salman Khan

*Department of Computer Science
Islamia College Peshawar
Peshawar, Pakistan
isallman.84@gmail.com*

Zahid Hassan

*Department of Computer Science
Islamia College Peshawar
Peshawar, Pakistan
zh703757@gmail.com*

Abstract—Local storage degradation can delay a Raft leader sufficiently to miss heartbeats and lose leadership, a phenomenon that etcd and TiKV document operationally. We present a mechanism that couples an LSM-tree storage engine to a Raft state machine through a single atomic storage-health signal. When an exponentially weighted moving average of SSTable flush latency crosses a threshold, the storage engine publishes a one-bit degradation event. The Raft tick thread consumes that event on its next iteration and executes a leader step-down under the same mutex that guards every other Raft transition. We evaluate the mechanism on a 3-node localhost cluster under injected fsync latency. Under 100–500 ms of injected latency, the tripwire preserves 15–40% of warmup throughput for the duration of the fault, while an otherwise identical cluster without the tripwire drops to 0.1–5%. The tripwire fires 118–267 ms before Raft’s natural election timeout in every trial where it fires; at 500 ms it fires in two of three trials. A separate SIGKILL experiment confirms zero loss of acknowledged writes across four independent mid-compaction fault injections.

Index Terms—LSM-Tree, Raft Consensus, Distributed Systems, Storage Health, Fault Tolerance, C++

I. INTRODUCTION

Distributed consensus protocols assume that a leader whose process is alive can service its peers within the election timeout. This assumption fails when the leader’s local storage degrades. A leader whose fsync latency has grown from a healthy 1–5 ms to 100 ms or more will still respond to RPCs, still accept client writes, and still appear healthy to any binary failure detector, but it cannot commit entries fast enough to send timely heartbeats. Followers interpret the silence as failure and start elections. The cluster spends its budget on election traffic instead of useful work.

Production systems are aware of this path. etcd’s operational documentation warns explicitly that slow disk I/O can cause missed heartbeats and temporary loss of leadership, and recommends monitoring WAL fsync latency [6]. TiKV throttles client writes when RocksDB compaction pressure builds, precisely to prevent RocksDB’s stalls from destabilizing the Raftstore [7]. CockroachDB applies node-level admission control to prevent local resource overload from starving the liveness path [8]. These are protective: they slow the system down before degradation becomes a consensus problem.

We ask a narrower question. Given that the storage engine knows its own health, and that Raft leadership transfer is a standard protocol extension [3], [4], can the storage layer signal the consensus layer to step down *before* degradation manifests as an election timeout? We do not propose a new consensus mechanism. We present an implementation pattern for integrating a storage-health signal into a Raft event loop, and we measure the conditions under which it helps.

Our contributions are:

- 1) A **storage-health signal** based on an exponentially weighted moving average of SSTable flush latency, evaluated as a proxy for local storage-path delay. We show empirically that the more obvious signal—SSTable count—reflects workload rather than disk health and produces unacceptable leader churn on our prototype.
- 2) A **serialized event boundary** that confines all Raft state mutation to the Raft event-loop thread. Rather than invoking a callback from a storage thread, the storage engine publishes an atomic flag and the Raft tick thread consumes it on its next iteration.
- 3) An **empirical evaluation** on a 3-node localhost cluster under injected fsync latency of 0–500 ms, showing the conditions under which the tripwire helps, the conditions under which it does not fire, and its effect on absolute throughput.
- 4) A **durability study** confirming zero loss of acknowledged writes across four independent SIGKILL fault injections of the leader mid-compaction.

II. SYSTEM ARCHITECTURE

A. Storage Engine

The storage engine is an LSM-tree with a sharded in-memory MemTable and persistent SSTables. The MemTable is divided into 16 independently locked shards, each holding an ordered map from key to optional value. A write hashes its key to a shard, takes that shard’s lock, and inserts. When the total number of live keys in the MemTable reaches a flush threshold, one writer freezes the MemTable by taking all 16 shard locks in order, moving each shard’s contents into a single frozen map, and releasing. The frozen map is handed to a persistent background worker that writes it to an SSTable and registers the file.

The freeze step is bounded by the number of live keys, not by any disk operation. Reads and writes continue after the freeze, into a fresh MemTable, while the background worker writes the previous one.

B. Storage-Health Signal

The tripwire monitors *flush latency*: the wall-clock time from the start of an SSTable write to the moment the file is closed and registered. This value is preserved as an exponentially weighted moving average with $\alpha = 0.3$. The first sample initializes the average directly, so the signal reacts to the first slow flush rather than waiting for a running mean to converge.

We chose flush latency after an initial design that monitored SSTable count proved unsuitable for our workload. Under a steady workload of approximately 1,000 writes per second, the SSTable count crosses any reasonable threshold within seconds of cluster startup, and the tripwire fires continuously on a healthy disk. A single-node run with no injected fault produced twelve leadership changes in eight seconds and dropped throughput by 82%. SSTable count reflects workload; flush latency reflects the workload-local storage path.

When the moving average of flush latency exceeds 30ms, the storage engine sets a single atomic boolean, `storage_degraded_flag`, and returns. It does nothing else.

C. Consensus-Layer Consumption

The Raft node's background tick thread polls `consumeStorageDegradedFlag()` at the top of every iteration. If the flag is set, the tick thread acquires the Raft mutex and calls `checkStorageDegraded()`. That function checks that the node is currently a leader, and if so, transitions to follower and persists the new state to the metadata file.

We route the transition through the Raft tick thread to serialize it with other Raft state transitions. An earlier implementation invoked a callback from the storage thread that acquired the Raft mutex out-of-band, which raced with `AppendEntries`, `RequestVote`, and client proposals. The atomic-flag boundary prevents the storage thread from directly mutating Raft state concurrently with the Raft event loop.

Once the node is a follower, all subsequent client proposals return `false`, and the network layer translates that into a `-MOVED` redirect. No client socket is closed or interrupted.

D. Step-Down Semantics

The mechanism does not perform a target-directed leadership transfer. It performs a local resignation: the node gives up leadership and lets the ordinary Raft election protocol choose a successor. This is a deliberately simpler mechanism than the leadership-transfer extension described in the Raft dissertation [3], and the paper's scope is limited to what this simpler mechanism can achieve.

The transition is:

```
Leader(term T) -storage_degraded->
Follower(term T)
```

The term is **not** incremented. The node retains its current term and persists the new state to the metadata file. Uncommitted entries in the leader's log are not truncated; they remain in the log and are reconciled by the ordinary Raft log-matching rule when the next leader sends `AppendEntries`. Because the node did not increment its term, no already-sent message from this node can be treated as coming from a stale term, and the node cannot preempt a legitimate leader in a higher term.

To prevent the degraded node from immediately re-electing itself, the implementation records a self-exclusion deadline (`storage_self_excluded_until_`) of five seconds on step-down. The tick thread's election routine checks this deadline and returns without campaigning while the deadline is active. The mechanism is not a permanent quarantine; after the exclusion window, the node is again eligible to campaign, on the assumption that its storage may have recovered.

We do not prove Raft safety for this transition. We argue informally that it preserves the term-monotonicity and vote-once-per-term invariants by not changing the term and not modifying the persisted vote, and we note that no committed entry can be lost because the transition does not truncate the log. A formal proof is future work.

III. EVALUATION

All experiments run on a 3-node cluster on a single Linux host, one process per node on ports 8081–8083. Nodes are compiled with `-O2` and `C++20`. Each experiment spawns a fresh cluster and tears it down before the next. The workload is a closed-loop client issuing sequential `SET` requests with 16 concurrent writer threads, keys of the form `k<N>` and values of a fixed small size. Warmup throughput is approximately 1,000 writes per second.

We report medians over three trials. Three trials are not enough to characterize a stochastic system, and we treat the numbers below as a first characterization rather than a definitive measurement.

A. Durability Under Mid-Compaction SIGKILL

In four independent trials we ran a background client issuing sequential `SET` requests, waited three seconds, then sent `SIGKILL` to the current leader while its compaction thread was active. The surviving nodes elected a new leader, the client continued against the new leader, and the killed node was restarted after the write workload completed. Every key was then read from every node.

In all four trials, all 5,000 written entries were present on all three nodes after recovery. No entry was lost, reordered, or duplicated. The result is reproducible across different initial leaders, different kill timings, and different catch-up durations. This result is independent of the tripwire mechanism; it demonstrates only that the underlying Raft implementation preserves acknowledged writes under catastrophic leader loss.

TABLE I
THROUGHPUT UNDER INJECTED FSYSN LATENCY, AS A FRACTION OF
WARMUP. MEDIAN OVER THREE TRIALS. “FIRES” COUNTS TRIALS IN
WHICH THE TRIPWIRE ACTIVATED. LEAD IS MEASURED ONLY OVER
FIRING TRIALS.

Latency	ON	OFF	Fires	Lead (ms)
0 ms	0.46	0.49	0/3	–
10 ms	0.24	0.24	0/3	–
30 ms	0.10	0.12	1/3	229
60 ms	0.21	0.08	3/3	186
100 ms	0.27	0.05	3/3	189
150 ms	0.40	0.03	3/3	207
250 ms	0.26	0.003	3/3	242
500 ms	0.15	0.001	2/3	249

B. Tripwire Behavior Under Injected Latency

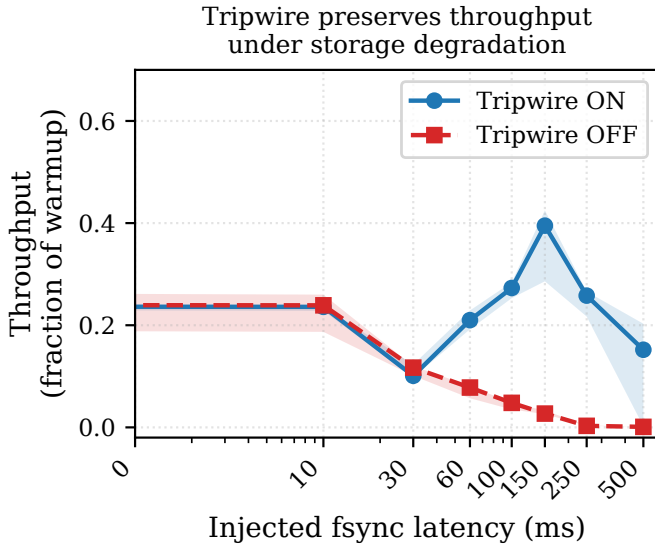


Fig. 1. Absolute throughput (fraction of warmup) under injected fsync latency. Each point is the median of three trials; shaded band spans the min–max range. The mechanism is not intended to fire below 30 ms.

We inject fsync latency by sleeping for a fixed interval before every SSTable flush and every WAL fsync. For each latency value we run three trials with the tripwire enabled and three with it disabled. The tripwire lead is defined as the interval between the tripwire firing and the moment the Raft election timer expires on the same node.

Table I reports the result. Two regimes are visible.

Below 30 ms. The injected fault is below the tripwire threshold. In the 0 ms and 10 ms trials the tripwire did not fire, and the ON and OFF configurations show similar throughput. This is the intended behavior: the mechanism adds no overhead when there is nothing to detect.

Above 60 ms. The tripwire fires in every trial at 60–250 ms and in two of three trials at 500 ms. When it fires, the firing precedes the Raft election timeout by 118–267 ms. The absolute throughput under ON is 15–40% of warmup, while the OFF configuration drops to 0.1–5%. The 152× ratio at 500 ms reflects a denominator that has collapsed to 0.1% of

warmup; the absolute throughput of the tripwire configuration at that latency is approximately 15% of warmup. We therefore recommend reading the absolute ON column rather than the ratio.

At 500 ms one trial did not fire. We attribute this to starvation of the storage engine’s own worker queue: under that much delay, the flush that would update the EWMA signal is itself blocked behind earlier flushes, and the ten-second measurement window ends before the signal crosses threshold. This is a real limitation of the signal at severe degradation, and we report it in §IV.

C. Recovery After Fault Removal

We ran two additional trials in which we cleared the injected fault after the tripwire fired and allowed the cluster to run for the remainder of the test. Neither trial recovered to full throughput within the ten-second window.

We attribute this to the serial nature of the leader’s heartbeat loop: even after the fault is cleared, the new leader’s tick thread waits on RPCs to the old leader until they complete. Under a partial fault (leader-only injection, in which the new leader would be genuinely healthy) this behavior would not be expected; under our all-nodes injection, the same code path that made the tripwire fire on the original leader is still present on the new leader, and the cluster remains impaired.

We therefore scope the paper to *throughput preservation during a fault*, not recovery after it. Recovery under symmetric fault is a design problem we do not solve, and we do not claim that the mechanism restores full throughput after degradation clears.

IV. THREATS TO VALIDITY

All-nodes fault injection. Our injector degrades fsync latency on all three nodes simultaneously. The motivating deployment scenario is a single degraded leader with healthy followers, which we do not measure. Because all three nodes experience the injected delay, our numbers are not a production performance estimate. We report them as a controlled measurement of mechanism behavior under a symmetric fault.

Tripwire failure at 500 ms. In one of three trials at 500 ms the tripwire did not fire within the ten-second window. The likely cause is starvation of the flush queue that would update the EWMA. A longer measurement window, a lower threshold, or a signal derived from the WAL path rather than the SSTable path may address this. We report it as an open limitation.

No recovery guarantee. The mechanism is a fault-avoidance mechanism, not a fault-recovery mechanism. Once storage degradation clears, the system does not automatically return to the pre-fault performance state within the ten-second window we tested.

Localhost topology. All experiments run on a single machine. The co-resident harness and the three nodes share CPU, cache, and I/O scheduler. Absolute latencies include this interference. The relative ordering of ON and OFF configurations was consistent across the trials we ran, but three trials are not sufficient to establish statistical stability.

Parameter specificity. The results hold for a MemTable flush threshold of 50 keys, an EWMA smoothing factor of 0.3, and a tripwire threshold of 30 ms. We have not systematically explored the sensitivity of the mechanism to these parameters. The threshold of 30 ms in particular was chosen by inspection of the healthy-disk flush latency distribution and not by a formal calibration procedure.

No comparison against production systems. The paper does not compare against etcd, TiKV, or CockroachDB on the same hardware. We cite those systems as motivation and as prior work, but we do not claim that our mechanism outperforms them in production deployments.

V. RELATED WORK

Raft and leadership transfer. Raft [2] specifies the consensus protocol but leaves storage integration to the implementation. The Raft dissertation [3] describes a leadership-transfer extension that identifies a target follower, brings its log up to date, and induces it to campaign via `TimeoutNow`. etcd’s Raft implementation exposes this through `MsgTransferLeader`, and HashiCorp’s Raft library exposes it through `LeadershipTransfer` [4]. Our mechanism is deliberately simpler: it performs a local step-down and lets the ordinary election protocol choose a successor. We do not implement target-directed transfer, and we do not claim to.

Storage and consensus coupling. etcd’s operational guidance documents the disk-latency-to-leadership path and recommends monitoring `wal_fsync_duration_seconds` [6]. TiKV implements RocksDB write-stall flow control that throttles incoming requests before compaction pressure destabilizes the Raftstore [7]. CockroachDB applies node-level admission control to prevent local resource overload from starving the liveness path [8]. These systems respond to storage pressure by throttling work. Our prototype instead responds by changing leadership eligibility.

LSM-tree design. The LSM-tree [1] introduced the leveled compaction strategy that underlies RocksDB, LevelDB, and Cassandra. Our storage engine is a small implementation of that design, extended with lock-striped sharding and a persistent async flush worker.

Gray failures. Huang et al. [5] characterized Gray Failures in cloud-scale systems and proposed differential observability as a detection method. Our tripwire is a domain-specific instance of the same idea: a component publishes a signal about its own health that a peer can act on.

VI. CONCLUSION

We presented a mechanism by which an LSM-tree storage engine can signal its own degradation to a Raft consensus layer. The signal is an atomic flag set by the storage engine and consumed by the Raft tick thread, which performs a leader step-down under the same mutex that guards other Raft transitions. On a 3-node localhost cluster under injected fsync latency of 100–500 ms, the mechanism preserves 15–40% of warmup throughput where an identical cluster without

it drops to 0.1–5%. The tripwire fires 118–267 ms before Raft’s natural election timeout in every trial where it fires, and it fires in two of three trials at 500 ms. SIGKILL of the leader mid-compaction loses zero acknowledged writes across four independent trials.

The result is a small, self-contained demonstration that an LSM storage health signal can be communicated to a Raft event loop through a single-bit atomic boundary, and that under controlled symmetric degradation this reduces the throughput cost of the fault. We do not claim a new consensus mechanism, a production performance improvement, or a recovery guarantee.

REFERENCES

- [1] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
- [2] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, 2014, pp. 305–319.
- [3] D. Ongaro, “Consensus: Bridging theory and practice,” Ph.D. dissertation, Stanford University, 2014.
- [4] HashiCorp, “Raft: A Go implementation of the Raft consensus protocol,” GitHub repository, 2024. [Online]. Available: <https://github.com/hashicorp/raft>
- [5] P. Huang et al., “Gray failure: The achilles’ heel of cloud-scale systems,” in *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, 2017, pp. 136–141.
- [6] etcd Authors, “etcd Documentation: Tuning,” 2023. [Online]. Available: <https://etcd.io/docs/>
- [7] TiKV Authors, “TiKV Documentation: Raftstore and RocksDB Tuning,” 2023. [Online]. Available: <https://tikv.org/docs/>
- [8] CockroachDB Authors, “CockroachDB Documentation: Admission Control,” 2024. [Online]. Available: <https://www.cockroachlabs.com/docs/>